

An introduction to R programming

Introduction

- R is a free, open-source statistical computing language and environment available for Windows, Mac OS X, Linux and Unix and licensed under the terms of the GNU (GNU is Not Unix) General Public License . This means anyone can see all its source code, and there are no restrictive, costly licensing arrangements
- R is an integrated suite of software facilities for data manipulation, calculation and graphical display. Among other things it has
 - An effective data handling and storage facility
 - A suite of operators for calculations on arrays, in particular matrices
 - A large, coherent, integrated collection of intermediate tools for data analysis
 - Graphical facilities for data analysis and display either directly at the computer or on hard copy
 - A well developed, simple and effective programming language which includes conditionals, loops, user defined recursive functions and input and output facilities.
- R is available on the Comprehensive R Archive Network (CRAN) website (<http://cran.r-project.org/>), and installation is straightforward.

R environment

- The term ‘environmental’ is intended to characterize it as a fully planned and coherent system, rather than an incremental accretion of very specific and inflexible tools, as is frequently the case with other data analysis software.
- R can be regarded as an implementation of the S language which was developed by Bell Laboratories by Rick Becker, John Chambers and Allen Wilks, and also forms the basis of the S-Plus systems.
- R initially created by Robert Gentleman and Robert Ihaka
- S as a language with three current implementations or “engines”, the “old S engine” (S version 3; S-Plus 3.x and 4.x), the “new S engine” (S version 4; S-Plus 5.x and above), and R
- R is a modern, functional programming language that allows for rapid development of ideas, together with object-oriented features for rigorous software development.

R vs S engines

- Lexical Scoping:
 - Contrary to other implementations of the S language, R has adopted an evaluation model in which nested function definitions are lexically scoped. This difference becomes manifest when *free* variables occur in a function. Free variables are those which are neither formal parameters (occurring in the argument list of the function) nor local variables (created by assigning to them in the body of the function).
 - In S, the values of free variables are determined by a set of global variables (similar to C, there is only local and global scope). In R, they are determined by the environment in which the function was created.

Example for Lexical scoping

- Consider the following function:

```
cube <- function(n) {  
  sq <- function() n * n  
  n * sq()  
}
```

- Under S, sq() does not “know” about the variable n unless it is defined globally:

```
S> cube(2)  
Error in sq(): Object "n" not found  
Dumped
```

```
S> n <- 3  
S> cube(2)  
[1] 18
```

- In R, the “environment” created when cube() was invoked n is also looked in:

```
R> cube(2)  
[1] 8
```

R vs S engines

- Models: There are some differences in the modeling code, such as
 - Whereas in S, you would use `lm(y ~ x^3)` to regress y on x^3 , in R, you have to insulate powers of numeric vectors (using `I()`), i.e., you have to use `lm(y ~ I(x^3))`.
 - The glm family objects are implemented differently in R and S. The same functionality is available but the components have different names.
 - Option `na.action` is set to "na.omit" by default in R, but not set in S.
 - Terms objects are stored differently. In S a terms object is an expression with attributes, in R it is a formula with attributes. The attributes have the same names but are mostly stored differently.
 - Finally, in R `y ~ x + 0` is an alternative to `y ~ x - 1` for specifying a model with no intercept. Models with no parameters at all can be specified by `y ~ 0`.

R and Object Orientation

- Packaging: a crucial infrastructure to efficiently produce, load and keep consistent software libraries from (many) different sources / authors
- Statistics: most packages deal with statistics and data analysis
- State of the art: many statistical researchers provide their methods as R packages
- primitive (or: atomic) data types in R are:
 - numeric (integer, double, complex)
 - character
 - logical
 - function
- out of these, vectors, arrays, lists can be built.

Object orientation

- Object: a collection of atomic variables and/or other objects that belong together
- Example: a microarray experiment
 - probe intensities
 - patient data (tissue location, diagnosis, follow-up)
 - gene data (sequence, IDs, annotation)

Parlance:

- class: the “abstract” definition of it
- object: a concrete instance
- method: other word for ‘function’
- slot: a component of an object

Object orientation

Advantages:

Encapsulation (can use the objects and methods someone else has written without having to care about the internals)

Inheritance (hierarchical organization of complexity)

What R does and does not?

- data handling and storage:
numeric, textual
- matrix algebra
- hash tables and regular expressions
- high-level data analytic and statistical functions
- classes (“OO”)
- graphics
- programming language:
loops, branching,
subroutines
- is not a database, but
connects to DBMSs
- has no graphical user
interfaces, but connects to
Java, TclTk
- language interpreter can be
very slow, but allows to call
own C/C++ code
- no spreadsheet view of
data, but connects to
Excel/MsOffice
- no professional /
commercial support

Installation

- Sources, binaries and documentation for R can be obtained via CRAN, the “Comprehensive R Archive Network” whose current members are listed at <http://cran.r-project.org/mirrors.html>.
- The simplest way is to download the most recent R-x.y.z.tar.gz file, and unpack it with **tar xvfz R-x.y.z.tar.gz** on systems that have GNU tar installed.
- On other systems you need at least to have the gzip program installed. Then you can use **gzip -dc R-x.y.z.tar.gz | tar xvf -**

```
cd R-x.y.x
make
make check
make install
```

- On type Ubuntu

```
$ sudo apt-get install r-base
```

Running R

- When working in UNIX or at a command line in Windows, the command 'R' can be used both for starting the main R program in the form `R [options] [<infile] [>outfile]`, or, via the R CMD interface, as a wrapper to various R tools (e.g., for processing files in R documentation format or manipulating add-on packages) which are not intended to be called “directly”.
- Commands can be typed directly at the prompt, indicated by the `>` symbol, or saved in an ASCII file using any text editor. Since R is an interpreter language, each line of code is processed as it is entered.

```
>R <enter>
```

- To quit the R program

```
>q()
```

Creating and managing R programs

- As a user, you have the choice of writing commands directly at the prompt or storing them in text files that can be sourced with .R as extension.
- This is essential for generating reproducible findings, an ethical imperative in medical research studies involving human subjects.

```
>print("Hello to R programming environment")  
Hello to R programming environment
```

- Save the same in hello.R then execute with

```
>source("hello.R")  
Hello to R programming environment
```

Getting help

- R has inbuilt help facility similar to the ‘man’ facility of UNIX. To get more information on any specific named function, for example `anova`, the command is

```
>help(anova)
```

An alternative is

```
>?anova
```

- For a feature specified by special characters, the argument must be enclosed in double or single quotes, making it a “character string”: this also necessary for a few words with syntactic meaning including `if`, `for` and `function`

```
>help("[[")
```

- On most R installations help is available in HTML format by running

```
>help.start()
```

R commands

- R is an expression language with a very simple syntax
- Commands are case sensitive
- Normally all alphanumeric symbols are allowed plus `'` and `_`, with the restriction that a name must start with `'` or a letter, and if it starts with `'` the second character must not be a digit.
- Commands are separated either by a semi-colon (`;`), or by a newline.
- Elementary commands can be grouped together into one compound expression by braces (`{` and `}`).
- Comments can be put almost anywhere, starting with `#` mark, everything to the end of the line is comment.
- If a command is not complete at the end of a line, R will give a different prompt, by default `+` on the second or subsequent lines and continue to read the input until the command is syntactically complete.

Data permanency and removing objects

- The entities that R creates and manipulates are known as objects. These may be variables, arrays of numbers, character strings, functions or more general structures built from such components.
- During R session, objects are created and stored by name. The R command `objects()` can be used to display the names of the objects which are currently stored within R.

```
> objects()      or      > ls()
```

- The collection of objects currently stored is called the workspace.
- To remove objects the function `rm` is available.

```
>rm(x,y,z,ink,junk,temp,foo,bar)
```

Data permanency and removing objects

- All objects created during an R sessions can be stored permanently in a file for use in the future R sessions. At the end of each R session you are given the opportunity to save all the currently available objects.
- If you indicate that you want to do this, the objects are written to a file called `‘.Rdata’` (`>save(x, file="filename.Rdata")`) to use the same `>load("filename.Rdata")` in the current directory, and the command lines used in the session are saved to a file called `‘.Rhistory’`
- When R is started later time from the same directory it reloads the workspace from this file. At the same time the associated commands history is reloaded.
- So, it is recommended to create or use separate working directories for analysis conducted with R.
- To know and to set the working directory from R environment type `>getwd()` and `> setwd("path_to_directory")`

Vectors

- **Vector:** an ordered collection of data of the same type

```
> a = c(1, 2, 3)
```

```
> a*2
```

```
[1] 2 4 6
```

- **Example:** the mean spot intensities of all 15488 spots on a chip:
a vector of 15488 numbers
- In R, a single number is the special case of a vector with 1 element.
- Other vector types: character strings, logical

Creating variables (vectors) and assignment

- Assignment operator (<-) stores the value (object) on the right side of (<-) expression in the left side.
- Once assigned, the object can be used just as an ordinary component of the computation. To find out what the object looks like, simply type its name. Note that R is case sensitive, e.g., object names abc, ABC, Abc are all different
- You can also use the "=" symbol or assign("a",3)
- **Important note:** since there are many built-in functions in R, make sure that the new object names you assign are not already used by the system. A simple way of checking this is to type in the name you want to use. If the system returns an error message telling you that such object is not found, it is safe to use the name.
- Example:

```
> a <- 3
> a
[1] 3
b <- sqrt(a*a+3)
>b
[1] 3.464102
>
```

Creating numerical vectors and arithmetic

```
> a <- c(3.1, 4.2, 2.7, 4.1) # Assign with "arrow" rather than "="  
> c(3.3, 3.4, 3.8) -> b # Can also assign pointing right  
> assign("c", c(a, 4.0, b)) # Or explicitly to a variable name  
> c # Concatenation "flattens" arguments  
[1] 3.1 4.2 2.7 4.1 4.0 3.3 3.4 3.8
```

```
> 1/c # Operate on each element of vector  
[1] 0.3225806 0.2380952 0.3703704 0.2439024 0.2500000  
     0.3030303 0.2941176 [8] 0.2631579
```

```
> a * b # Cycle shorter vector "b" (but warn)  
[1] 10.23 14.28 10.26 13.53  
Warning message: longer object length is not a multiple of  
shorter object length in: a * b
```

```
> a+1 # "1" is treated as vector length 1  
[1] 4.1 5.2 3.7 5.1
```

Sorting and finding length, max & min of vectors

- Function `length()` can be used to find the length of the vector

```
> length(a)
```

- Function `sort()` can be used to sort the vector

```
> sort(a)
```

- Function `max()` can be used to find the maximum value in the vector

```
> max(a)
```

- Function `min()` can be used to find the minimum value in the vector

```
> min(a)
```

Generating regular sequences

- The function `seq()` is more general facility for generating sequences. It has five arguments, only some of which may be specified in any one call.
- The first two arguments, if given, specify the beginning and end of the sequence, and if these are the only two arguments given the result is the same as the colon `(:)` operator.
- The first two parameters may be named `from=value` and `to=value`; thus `seq(1:30)`, `seq(from=1, to=30)` and `seq(to=30, from=1)` are all same as `1:30`.
- The next two parameters to `seq()` may be `by=value` and `length=value`, which specify a step size and a length for the sequence respectively. If neither of these is given, the default `by=1` is assumed.

Generating regular sequences

```
> s3 <- seq(-5, 5, by=0.5)
```

```
> s4 <- seq(length=51, from=-5, by=0.2)
```

- A related function is `rep()` can be used for replicating an object in various complicated ways. The simplest form is

```
> s5 <- rep(s3, times=5)
```

- Following is another useful version , Which repeats each element of x five times before moving on to the next.

```
> s6 <- rep(s3, each=5)
```

Logical vectors

- Logical vectors are generated by *conditions*. For example

```
> x <- 13
```

```
> Temp <- x > 14
```

Sets temp as vector of the same length as x with values FALSE corresponding to elements of x where the condition is not met and TRUE where it is.

- The logical operators are <, <=, >, >=, == for exact equality and != for inequality.
- Logical vectors may be used in ordinary arithmetic, in which case they are coerced in to numeric vectors , FALSE becoming 0 and TRUE becoming 1.

Missing values

- In some cases the components of a vector may not be completely known. When an element or value is “not available” or a “missing value” in the statistical sense, a place within a vector may be reserved for it by assigning it the special value NA. In general any operation on an NA becomes an NA.
- The function `is.na(x)` gives the logical vector of the same size as `x` with value TRUE if and only if the corresponding element in `x` is NA.

```
> z <- c(1:4, NA); ind <- is.na(z)
```

- Note that there is second kind of “missing values” which are produced by numerical computation, the so-called “Not a Number” or NaN. For Example

```
> 0/0
```

- In summary, `is.na(x)` is TRUE both for NA and NaN values. To differentiate these `is.nan(x)` is only TRUE for NaN's.

Character Vectors

- Character quantities and character vectors are used frequently in R, for example as plot labels.
- Character strings are entered using either double quotes ("") or single ('), but are printed using double quotes.
- The paste() function takes an arbitrary number of arguments and concatenates them one by one into character strings. Any number of arguments and concatenates them one by one into character strings. The arguments are by default separated in the result by a single blank character, but this can be changed by named parameter, *sep=string*, which changes it to *string*, possibly empty.

```
>labs <- paste(c("X", "Y"), 1:5, sep="")
```

Selecting and modifying subsets of a data set

- Subsets of the elements of a vector may be selected by appending to the name of the vector an *index vector* in square brackets.
- Such index vectors can be any of four distinct types.

- **A logical vector:** In this case the index vector must be same length as the vector from which elements are to be selected. Values corresponding to TRUE in the index vector are selected and those corresponding to FALSE are omitted.

```
> y <- x[!is.na(x)]
```

```
> x[(!is.na(x) & x>3)] ->z
```

- **A vector of positive integral quantities:** In this case the values in the index vector must lie in the set {1,2,....., length(x)}. The corresponding elements of the vector are selected and concatenated, in that order, in the result.

```
> x[1:10] # will select first 10 elements of x
```

```
> c("x", "y")[rep(c(1,2,2,1), times=4)]
```

Selecting and modifying subsets of a data set

- **A vector of negative integral quantities:** Such an index vector specifies the values to be excluded rather than included. Thus

```
> y <- x[-(1:5)]
```

Gives y all except the first five element of x

- **A vector of character strings:** This possibility only applies where an object has *names* attributes to identify its components. In this case a sub-vector of the names vector may be used in the same way as the positives integral.

```
> fruit <- c(5, 10, 1, 20)
```

```
> names(fruit) <- c("Orange", "Banana", "Apple",  
"Peach")
```

```
> lunch <- fruit[c("Apple", "Orange")]
```

Other object types

- Vectors are the most important type of object in R, but there are several others. Including
 - Matrices
 - Factors
 - Lists
 - Data frames
 - Functions

Vector vs Lists

- **Vector:** an ordered collection of data of the same type.

```
> a = c(7, 5, 1)
```

```
> a[2]
```

```
[1] 5
```

- **List:** an ordered collection of data of arbitrary types.

```
> doe = list(name="john", age=28, married=F)
```

```
> doe$name
```

```
[1] "john"
```

```
> doe$age
```

```
[1] 28
```

- Typically, vector elements are accessed by their index (an integer), list elements by their name (a character string). But both types support both access methods.

Factors

- A good deal of statistical data is of a form which indicates which one of several possible categories that an observation falls into.
- Examples
 - Eye colour: brown, hazel, green, blue.
 - Location: North Island, South Island, other.
 - Pain level: low, medium, high.
- R provides a facility for creating this kind of data through the functions *factor()* and *ordered()*.
- The function *factor* creates data objects which represent variables containing unordered categorical data. It takes a character or numeric vector as an argument and returns a factor.

Factors

```
> eyes <- c("hazel", "blue", "brown", "green", "blue", "brown")
> eyecol <- factor(eyes)
> eyes
[1] "hazel" "blue" "brown" "green" "blue"  "brown"
> eyecol
[1] hazel blue brown green blue brown
Levels: blue brown green hazel
```

Factor Levels:

- The set of all possible categories for a set of categorical variable is called the levels of the corresponding factor. By default, R takes the levels to be the set of values occurring in the input data vector, sorted into ascending order (either numerically or alphabetically). When a factor is printed, the levels of the factor are displayed after the variable.

```
> eyecol
[1] hazel blue brown green blue brown
Levels: blue brown green hazel
```

Specifying Factor Levels

- The default set of factor levels and the order they appear in can be specified with a second argument to factor.

```
> eyecol = factor(eyes, levels = c("blue", "green",  
  "hazel", "brown"))
```

```
> eyecol
```

```
[1] hazel blue brown green blue brown
```

```
Levels: blue green hazel brown
```

- The levels of a factor can be obtained with the levels function.

```
> levels(eyecol)
```

```
[1] "blue" "green" "hazel" "brown"
```

- Function gl() can be used to generate factors by specifying the pattern of their levels

```
> gl(2, 8, labels=c("Control", "Treated"))
```

Ordered factors

- Sometimes there is a natural order to a factor's levels. In this case factors are described as ordered factors.
- Ordered factors are created with the R function *ordered()*.
- It is important to specify the levels when creating an ordered factor to ensure that they are in the correct relationship to each other.

```
> pain = ordered(c("low", "medium",  
  "medium", "high", "medium", "low"), levels =  
  c("low", "medium", "high"))
```

Factors and vectors

- Factors look very much like vectors, and for many purposes they can be treated as vectors. In fact, they are special kinds of objects which are created from vectors in a similar way to which matrices are created from vectors.
- To tell whether a value is a factor, use the functions, `is.factor` and `is.ordered`.

```
> is.factor(eyecol)
```

```
[1] TRUE
```

```
> is.factor(pain)
```

```
[1] TRUE
```

```
> is.ordered(eyecol)
```

```
[1] FALSE
```

```
> is.ordered(pain)
```

```
[1] TRUE
```

Operations on Factors

- About the only operation which makes sense with an unordered vector is to compare the values with a particular value using `==` or `!=`. For ordered factors, comparisons using `<`, `<=`, `>` and `>=` also make sense.

```
> eyecol == "blue"
```

```
[1] FALSE TRUE FALSE FALSE TRUE FALSE
```

```
> eyecol < "blue"
```

```
[1] NA NA NA NA NA NA
```

Warning message:

```
In Ops.factor(eyecol, "blue") : < not  
meaningful for factors
```

Tabulation with factors

- One of the few things that can be done with factors is to count the number of times each level occurs. This can be done with the function `table`.

```
> table(eyecol)
eyecol
blue green hazel brown
2      1      1      2
> table(pain)
pain
low medium high
2  3      1
> table(eyecol, pain)
      pain
eyecol low medium high
blue  0  2  0
green 0  0  1
hazel 1  0  0
brown 1  1  0
```

- The resulting matrix (or more general array) is called a contingency table.

Matrices and Arrays

- **Matrix:** A matrix refers to a numeric array of rows and columns. One of the easiest ways to create a matrix is to combine vectors of equal length using `cbind()`, meaning "column bind":
- **Example:** the expression values for 10000 genes for 30 tissue biopsies: a matrix with 10000 rows and 30 columns.
- **Array:** 3-,4-,...dimensional matrix
- **Example:** the red and green foreground and background values for 20000 spots on 120 chips: a 4 x 20000 x 120 (3D) array.

Creating and operating matrices

```
> x <- c(1,3,2,10,5)
> y <- c(1,2,3,4,5)
> m1<-cbind(x,y);m1
```

```
      x y
[1,]  1 1
[2,]  3 2
[3,]  2 3
[4,] 10 4
[5,]  5 5
```

- To transpose the matrix

```
> t(m1)                                # transpose of m1
      [,1] [,2] [,3] [,4] [,5]
x         1     3     2    10     5
y         1     2     3     4     5
```

Operating with matrices

- To find the dimension of the matrix

```
> dim(m1)           # 2 by 5 matrix
[1] 2 5
```

- Use `rbind()` to bind the rows

```
> m1<-rbind(x,y)      # rbind() is for row bind and
  equivalent to t(cbind()).
```

- Of course you can directly list the elements and specify the matrix using `matrix()` function:

```
> m2<-matrix(c(1,3,2,5,-1,2,2,3,9),nrow=3);m2
      [,1] [,2] [,3]
[1,]     1     5     2
[2,]     3    -1     3
[3,]     2     2     9
```

- Note that the elements are used to fill the first column, then the second column and so on. To fill row-wise, we specify `byrow=T` option.

Indexing matrices

- Extracting the component of a matrix involves one or two indices.

```
> m2[2,3]           #element of m2 at the second row, third
  column
```

```
[1] 3
```

```
> m2[2,]           #second row
```

```
[1]  5 -1  2
```

```
> m2[,3]           #third column
```

```
[1] 2 2 9
```

```
> m2[-1,]          #submatrix of m2 without the first row
```

```
  [,1] [,2] [,3]
```

```
[1,]    5   -1    2
```

```
[2,]    2    3    9
```

```
> m2[-1,-1]        #submatrix of m2 with the first row and column
  removed
```

```
  [,1] [,2]
```

```
[1,]   -1    2
```

```
[2,]    3    9
```

Matrix calculation

- Matrix computation is usually done component-wise.

```
> m1<-matrix(1:4, ncol=2); m2<-matrix(c(10,20,30,40),ncol=2)
> 2*m1                      # scalar multiplication
> m1+m2                      # matrix addition
> m1*m2                      # component-wise multiplication
```

- Note that $m1*m2$ is NOT the usual matrix multiplication. To do the matrix multiplication, you should use `%*%` operator instead.

```
> m1 %*% m2
      [,1] [,2]
[1,]    70  150
[2,]   100  220
```

```
> solve(m1)                  #inverse matrix of m1
      [,1] [,2]
[1,]   -2  1.5
[2,]    1 -0.5
> solve(m1)%*%m1             #check if it is so
```

Matrix calculation

```
> diag(3)                                #diag() is used to construct a k by k
  identity matrix
      [,1] [,2] [,3]
[1,]     1     0     0
[2,]     0     1     0
[3,]     0     0     1

> diag(c(2,3,3))                         #as well as other diagonal matrices
      [,1] [,2] [,3]
[1,]     2     0     0
[2,]     0     3     0
[3,]     0     0     3
```

- **Eigenvalues and eigenvectors of a matrix is handled by eigen() function:**

```
> eigen(m2)
$values
[1] 53.722813 -3.722813

$vectors
      [,1]      [,2]
[1,] -0.5657675 -0.9093767
[2,] -0.8245648  0.4159736
```

Arrays

- Arrays can be constructed from vectors by the `arrays()` function, which has following form

```
> Z <- array(data_vector, dim_vector)
```

For example:

```
> h <- c(1:20)
```

```
> Z <- array(h, dim=c(3,4,2))
```

- However if `h` is shorter than 24, its value are recycled from the beginning again to make it up to size 24.
- At this point `dim(Z)` stands for the dimension of the vector `c(3,4,2)`, and `Z[1:24]` (`as.vector(Z)` or `c(Z)`) stands for the entire array as it was in `h`.

Data frames

- Data frames provide a way of grouping a number of related variables into a single data object. The function `data.frame` takes a number of vectors and/or factors and returns a single object containing all the variables.

```
df = data.frame(var1, var2, ...)
```

- Each of the `var1, var2, ...` is either an expression specifying a vector or factor, or a named expression of the form

```
name = var
```

- where `name` provides a name for the given variable in the data frame.
- It is a rectangular table with rows and columns; data within each column has the same type (e.g. number, text, logical), but different columns may have different types.

Creating Data frame

- A simple gender/height data set.

```
> sex = factor(rep(c("female", "male"), c(4, 4)))
> hgt = c(165, 176, 171, 177, 176, 193, 180, 193)
> classinfo = data.frame(sex, hgt)
> classinfo
```

	sex	hgt
1	female	165
2	female	176
3	female	171
4	female	177
5	male	176
6	male	193
7	male	180
8	male	193

Subsetting Data frame

- Individual elements of a vector, matrix, array or data frame are accessed with “[]” by specifying their index, or their name

```
> localisation <- c('proximal', 'distal', 'proximal')
> tumorsize <- c(6.3, 8.0, 10.0); progress <- c(0, 1, 0)
> a <- data.frame(localisation, tumorsize, progress,
row.names=c("XX348", "XX234", "XX987"))
```

```
> a
```

	localisation	tumorsize	progress
XX348	proximal	6.3	0
XX234	distal	8.0	1
XX987	proximal	10.0	0

```
> a[3, 2]
```

```
[1] 10
```

```
> a["XX987", "tumorsize"]
```

```
[1] 10
```

```
> a["XX987",]
```

	localisation	tumorsize	progress
XX987	proximal	10	0

Updating Data frame

- The function `transform` can be used to produce new variables from those already present in a data frame or to add new variable and to combine all the variables into a new data frame.

```
> nclass = transform(classinfo, hgt2 = hgt^2)
```

```
> nclass
```

```
sex hgt hgt2
```

```
1 female 165 27225
```

```
2 female 176 30976
```

```
3 female 171 29241
```

```
4 female 177 31329
```

```
5 male 176 30976
```

```
6 male 193 37249
```

```
7 male 180 32400
```

```
8 male 193 37249
```

> a

	localisation	tumorsize	progress
XX348	proximal	6.3	0
XX234	distal	8.0	1
XX987	proximal	10.0	0

> a[c(1,3),]

	localisation	tumorsize	progress
XX348	proximal	6.3	0
XX987	proximal	10.0	0

> a[c(T,F,T),]

	localisation	tumorsize	progress
XX348	proximal	6.3	0
XX987	proximal	10.0	0

> a\$localisation

[1] "proximal" "distal" "proximal"

> a\$localisation=="proximal"

[1] TRUE FALSE TRUE

> a[a\$localisation=="proximal",]

	localisation	tumorsize	progress
XX348	proximal	6.3	0
XX987	proximal	10.0	0

Subsetting

subset rows by a
vector of indices

subset rows by a
logical vector

subset a column

comparison resulting in
logical vector

subset the selected
rows

Grouping, conditional execution , and loops

- Grouped expressions

- R is an expression language in the sense that its only command type is a function or expression which returns a result.
- Commands may be grouped together in braces, {expr 1, . . . , expr m}, in which case the value of the group is the result of the last expression in the group evaluated.

- Control statements

- if statements
- The language has available a conditional construction of the form
if (expr 1) expr 2/statments else expr 3
where expr 1 must evaluate to a logical value and the result of the entire expression is then evident.
- a vectorized version of the if/else construct, the ifelse function. This has the form ifelse(condition, a, b)

Operators

- Comparison operators:
 - == (equal)
 - != (not equal)
 - >= (greater than or equal)
- Logical operators:
 - & (and)
 - | (or)
 - ! (not).

If....else

1. If Statement operates on length-one logical vectors

Syntax:

```
if (cond1=true) { cmd1 } else { cmd2 }
```

Example:

```
if (1==0) { print(1) } else { print(2) }
```

- Avoid using newlines between '}' else'. To apply if statement on many values of vector or data frame, use 'for' or 'apply' loops (see below).

2. Ifelse Statement: operates on vectors

Syntax:

```
ifelse(test, true_value, false_value)
```

Example:

```
x <- 1:10
```

```
ifelse(x<5 | x>8, x, 0)
```

Example: if.. else

```
> grade = "PASS" # default value for grade
> score = 50 # actual score
> if (score<60) grade = "FAIL"
> grade # actual grade

> score<-50
> if (score<60) grade = "FAIL" else grade = "PASS"
> grade
```

OR

```
> score<-50
> grade <- ifelse(score<60, "FAIL","PASS")
> grade
```

Switch statements

- When there are many conditions to evaluate, the if-else statement can become complex and verbose. A much cleaner solution is the switch statement. The switch statement allows testing any integral value or string against multiple values. When the test produces a match, all statements associated with that match are executed.

```
> days <- c("Monday", "Sunday", "Wednesday", "Friday",  
  "Saturday", "Tuesday", "Thursday")  
> for(day in days) {  
+   cat(day, ":",  
+   switch(EXPR=day, Monday="I go to movie", Sunday="It  
  will rain", Saturday="I go to temple", Wednesday="I  
  go to my friends", "No match!!"), "\n"  
+ )  
+ }
```

Loops

- When the same or similar tasks need to be performed multiple times; for all elements of a list; for all columns of an array; etc. loops will help.
- The most commonly used loop structures in R are 'for', 'while' and 'apply' loops. Less common are 'repeat' loops.
- The 'break' function is used to break out of loops, and 'next' halts the processing of the current iteration and advances the looping index.

For loops

- For Loop: flexible, but slow when looping over large number of fields (e.g. thousands of rows or columns)

Syntax:

```
for(variable in sequence) {  
    statements  
}
```

Example:

```
> for(i in 1:10) {  
    print(i*i)  
}
```

More on for loops

Example: condition*

```
x <- 1:10; z <- NULL
for(i in x) {
  if (x[i]<5) { z <- c(z,x[i]-1) } else { z <-
    c(z,x[i]/x[i]) }
}
```

Example: stop on condition and print error message

```
x <- 1:10; z <- NULL
for (i in x) {
  if (x[i]<5) { z <- c(z,x[i]-1) }
  else { stop("values need to be <5")}
}
```

While loop

- Similar to for loop, but the iterations are controlled by a conditional statement.

Syntax:

```
while(condition) statements
```

- Loop continues until condition returns FALSE.

Example:

```
> i=1
> while(i<=10) {
    print(i)
    i=i+sqrt(i)
}
```

Apply family loops

- Apply the function along some dimensions of the array, according to margin, returns a vector or array of the appropriate size.

Syntax: `apply(X, MARGIN, FUN, ARGS)` where, X: array, matrix or data.frame; MARGIN: 1 for rows, 2 for columns, c(1,2) for both; FUN: one or more functions; ARGS: possible arguments for function

Example: `apply(my_matrix, 1, function(i) { my_commands } )`

```
> x
```

```
      [,1] [,2] [,3]
[1,]     5     7     0
[2,]     7     9     8
[3,]     4     6     7
[4,]     6     3     5
```

```
> apply(x, 1, sum)
```

```
[1] 12 24 17 14
```

```
> apply(x, 2, sum)
```

```
[1] 22 25 20
```

Lapply loop

- When the same or similar tasks need to be performed multiple times for all elements of a list or for all columns of an array.
- May be easier and faster than “for” loops
- Syntax: `lapply(li, function )`
 - To each element of the list `li`, the function *function* is applied.
 - The result is a list whose elements are the individual *function* results.

```
> li = list("klaus", "martin", "georg")
> lapply(li, toupper)
> [[1]]
> [1] "KLAUS"
> [[2]]
> [1] "MARTIN"
> [[3]]
> [1] "GEORG"
```

Supply loop

- Syntax: `sapply( li, fct )`
- Like `apply`, but tries to simplify the result, by converting it into a vector or array of appropriate size

```
> li = list("klaus", "martin", "georg")
> sapply(li, toupper)
[1] "KLAUS"  "MARTIN" "GEORG"
```

```
> fct = function(x) { return(c(x, x*x, x*x*x)) }
> sapply(1:5, fct)
      [,1] [,2] [,3] [,4] [,5]
[1,]    1    2    3    4    5
[2,]    1    4    9   16   25
[3,]    1    8   27   64  125
```

Tapply loops

- Syntax: `tapply(vector, index, Fun)`
- The arguments are a vector, a list of factors and a function that operates on a vector to return a single value. For each combination of factor levels, the function is applied to corresponding values of the variable.
- The `tapply` function is useful when we need to break up a vector into groups defined by some classifying factor, compute a function on the subsets, and return the results in a convenient form.

- **Example:**

```
> medicalExp <- data.frame(patient=1:10, age=c(53,54, 56,
23,    45,    56,    48,    39,23),    treatment=    gl(2,5,
labels=c("Treatment", "Control")))
```

```
> tapply(medicalExp$age, medicalExp$treatment, mean)
```

Treatment	Control
48.6	42.2

Repeat loops

- Syntax:

repeat statement

- Loop is repeated until a break is specified. This means there needs to be a second statement to test whether or not to break from the loop.
- Example

```
> repeat {  
  z <- 2;  
  z <- z+2;  
  print(z);  
  z <- z+6;  
  print(z);  
  break  
}
```

```
#A program for  
multiplication table  
> i <- 0  
> x <- 5  
> repeat {  
  i = i+1  
  mult <- x * i  
  print(mult)  
  if(i==20){  
    break;  
  }  
}
```

Reading data from files

- The standard way of storing statistical data is to store them in a rectangular form with rows corresponding to observations and columns corresponding to variables.
- Spreadsheets are often used to store and manipulate data in this way.
- The function `read.table` can be used to read data which has been stored in this way.
- The first argument to `read.table` identifies the file to be read.

Reading data from file: File Names and Path Names

- A simple file name (i.e. one not containing / in Linux or \ in Windows) identifies a file in directory where R was invoked.
- Files in other places can be specified with a relative or absolute path name.
- In Linux, path names have the form: `dir/.../dir/file`
- where `dir` is a directory name and `file` is a file name.
 - A leading / indicates the top level of the file system.
 - A leading ~ indicates the user's home directory.
 - The character . represents the current directory.
 - The character .. represents the directory one level up from the current one.
- The specification `../data/foo.txt` specifies the file called `foo.txt` which can be located by going up one level and then down into a directory called `data`.

Customized Use of read.table

- There are many optional arguments to read.table which can be used to change its behaviour.
 - Setting header=TRUE indicates to R that the first row of the data file contains names for each of the columns.
 - The argument skip= makes it possible to skip the specified number of lines at the top of the file (for cases where there might be documentation or metadata at the top of the file).
 - The argument sep= can be used to specify a character which separates columns (the default is to take white space as the column separator).
 - The argument row.names= specifies row names. This is either a vector of row names, a positive integer specifying which column contains the row names, or the variable name corresponding to the row.names.

Customized Use of read.table

- The argument `col.names` specifies a vector of names for the columns (for the case where `header=FALSE`. The names default to V followed by the column number.
- The argument `as.is` controls the conversion of variables to factors (the default behaviour is to convert character variables to factors). This can either be a vector of logicals specifying whether the corresponding columns should be left unconverted, or a vector of the integer indices of the columns to be left alone.
- More flexibly, the argument `colClasses=` makes it possible to precisely specify the type of the data in the corresponding columns.

Reading data from file: example

```
> info <- read.table(file="geneInfo.txt", sep="\t", header=T,  
  row.names=1)
```

```
> info
```

	sample1	sample2	sample3	sample4	sample5	sample6	sample7
ZHX2	6.27	1.30	6.500	2.121	1.747	2.284	1.690
MAN1A1	6.10	6.50	1.400	6.300	2.029	1.642	2.176
N4BP1	6.60	6.90	6.400	3.400	1.993	1.603	2.130
EP350	2.30	-1.01	4.200	-1.010	-2.050	-1.056	-2.340
MDS2	3.40	4.44	1.234	1.500	1.600	1.300	1.200
C1orf53	2.34	1.25	2.333	2.300	2.600	1.200	3.100
STMN	-1.03	-3.20	-2.300	-2.620	-1.450	-1.670	-2.220

- There are number of variants of read.table which have slightly different behavior
 - read.csv assumes that (by default) columns are separated by commas
 - Read.csv2 assumes that (by default) columns are separated by semicolons and that the decimal indicator is a comma.

Playing around read data

```
> apply(info,1,mean)
      ZHX2      MAN1A1      N4BP1      EP350      MDS2      C1orf53      STMN
3.130286  3.735286  4.146571 -0.138000  2.096286  2.160429 -2.070000
> apply(info,2,mean)
      sample1      sample2      sample3      sample4      sample5      sample6      sample7
3.7114286  2.3114286  2.8238571  1.7130000  0.9241429  0.7575714  0.8194286
> meanexprsval <- apply(info,1,mean)
> o <- order(meanexprsval,decreasing=TRUE)
> o
[1] 3 2 1 6 5 4 7
> o <- sort(meanexprsval,decreasing=TRUE)
> o
      N4BP1      MAN1A1      ZHX2      C1orf53      MDS2      EP350      STMN
4.146571  3.735286  3.130286  2.160429  2.096286 -0.138000 -2.070000
> meanexprsval > 0
      ZHX2      MAN1A1      N4BP1      EP350      MDS2      C1orf53      STMN
      TRUE      TRUE      TRUE      FALSE      TRUE      TRUE      FALSE
> info[meanexprsval > 0,]
      sample1 sample2 sample3 sample4 sample5 sample6 sample7
ZHX2      6.27      1.30      6.500      2.121      1.747      2.284      1.690
MAN1A1      6.10      6.50      1.400      6.300      2.029      1.642      2.176
N4BP1      6.60      6.90      6.400      3.400      1.993      1.603      2.130
MDS2      3.40      4.44      1.234      1.500      1.600      1.300      1.200
C1orf53      2.34      1.25      2.333      2.300      2.600      1.200      3.100
```

Writing data to file

```
> write.table(info, file="geneInfo1.txt",  
  sep="\t")
```

Will create file with geneInfo1.txt

- R offers another way of storing data with commands “save” and “load”

```
> save(x, file="geneInfo2.Rdata")
```

```
> load("geneInfo2.Rdata")
```

Creating functions

- A function is defined by using the keyword **function**, followed by **an** opening parenthesis, a list of formal arguments (separated by commas), and a closing parenthesis, and then by the expressions that form the body of the functions.

```
> sq1<-function(x) x*x
```

```
> sq1(5)
```

```
[1] 25
```

- The value returned by a function is either the value that is explicitly returned by a call “return” or it is simply the value of the last expression.

```
> sq3<-function(x) {
```

```
+ y<-x*x
```

```
+ return(y)
```

```
+ }
```

```
> sq3(5)
```

Recursive function

- A recursive functions is function that calls itself.
- Consider computing a factorial: $n! = n * (n-1)!$
- A recursive function can be used here, because, to compute $n!$, one can compute $(n - 1)!$, and then multiplied by n .
- Algorithm:

- $0! = 1$
- $1! = 1 * 0! = 1 * 1 = 1$
- $2! = 2 * 1! = 1 * 1 = 2$
- $3! = 3 * 2! = 3 * 2 = 6$
- $4! = 4 * 3! = 4 * 6 = 24$

```
factorial<-function (n) {  
  If (n==0) return(1)  
  else return(n*factorial(n-1))  
}
```

```
> source(''factorial.R'')  
> factorial(0)  
> factorial(1)  
> factorial(3)
```

Packages

- A package is a standardized collection of material extending R
- Contain R functions, data sets, compiled dll's
- A default R installation contains several packages.
- Additional packages can be installed.
- Not part of the user workspace
- Can be loaded with a library statement

```
> library("genetics")
```

Graphical procedures

- Graphical facilities are an important and extremely versatile component of the R environment.
- Important plotting commands are as follows

```
> plot(x, y)
```

If x and y are vectors, `plot(x,y)` produces a scatter plot of y against x .

```
> plot(x)
```

If x is a time series, this produces a time-series plot. If x is a numeric vector, it produces a plot of the values against their index in the vector.

```
> plot(f, y)
```

Where f is factor object, y is a numeric vector. The first form generates a bar plot of f ; the second form produces boxplots of y for each level of f .

Running web application in R

- Example for obtaining MW values for peptide sequences from the EXPASY's pI/MW Tool web page.

```
myentries <- c("MKWVTFISLLFLFSSAYS", "MWVTFISLL",
"MFISLLFLFSSAYS")
myresult <- NULL
for(i in myentries) {
  myurl <- paste("http://ca.expasy.org/cgi-
bin/pi_tool?protein=", i, "&resolution=monoisotopic", sep="")
  x <- url(myurl)
  res <- readLines(x)
  close(x)
  mylines <- res[grep("Theoretical pI/Mw:", res)]
  myresult <- c(myresult, as.numeric(gsub(".* / ",
"", mylines)))
  print(myresult)
  Sys.sleep(1) # halts process for one sec to give
web service a break
}
final <- data.frame(Pep=myentries, MW=myresult)
cat("\n The MW values for my peptides are:\n")
print(final)
```